

Aula 06 – CSS3: Estilização Fundamental

1. Abertura da Aula

Até este momento, vocês aprenderam a estruturar informações na web utilizando HTML, criando o "esqueleto" das nossas páginas. No entanto, um sistema apenas com HTML puro é visualmente desinteressante e, muitas vezes, difícil de navegar. O problema que resolveremos hoje é justamente este: como transformar uma estrutura básica de dados em uma interface amigável, atraente e profissional. No contexto da nossa Situação de Aprendizagem, cujo foco é o "Desenvolvimento e Implantação de Soluções Digitais Inovadoras", o produto final precisa não apenas funcionar, mas também garantir a usabilidade e qualidade visual. No mercado de tecnologia, dominar o CSS é essencial, pois o Front-end é a ponte direta entre o seu sistema e o usuário final; um design bem aplicado aumenta a retenção de clientes e transmite credibilidade para qualquer aplicação web.

2. Conceitos Fundamentais

2.1. Seletores, Cascata e Especificidade

1. Explicação teórica

O CSS (Cascading Style Sheets) funciona selecionando elementos do HTML e aplicando regras visuais a eles. Para escolhermos *quem* será estilizado, utilizamos os **seletores**. Podemos selecionar um elemento pela sua tag (ex: `p`, `h1`), pela sua classe (utilizando um ponto, ex: `.botao`) ou pelo seu ID (utilizando a hashtag, ex: `#cabecalho`). O CSS é regido pelo conceito de **Cascata**, o que significa que, se duas regras com o mesmo peso apontarem para o mesmo elemento, a regra lida por último no código será a aplicada. Já a **Especificidade** é o sistema de pontuação do CSS para resolver conflitos de regras: um ID é mais forte que uma classe, e uma classe é mais forte que uma tag. Portanto, uma regra aplicada a um ID sobrescreverá uma regra aplicada a uma tag, independentemente da ordem em que aparecem.

2. Demonstração prática

CSS

```
/* Seletor de Tag - Especificidade Baixa */
```

```
p {  
  color: black;  
}
```

```
/* Seletor de Classe - Especificidade Média */
```

```
.texto-destaque {  
  color: blue;  
}
```

```
/* Seletor de ID - Especificidade Alta */
```

```
#alerta-erro {  
  color: red;  
}
```

Explicação linha por linha:

- Linha 2-4: Seleciona todos os parágrafos (`<p>`) da página e define a cor da fonte como preta.
- Linha 7-9: Seleciona qualquer elemento que contenha o atributo `class="texto-destaque"` e aplica a cor azul. Mesmo que seja um parágrafo, essa regra vence a da linha 2.
- Linha 12-14: Seleciona o elemento único que possui `id="alerta-erro"` e aplica a cor vermelha. Esta é a regra mais forte das três.

3. Exercício guiado

Abra o seu editor de código e crie um arquivo HTML contendo um título `<h1>` e dois parágrafos `<p>`. Adicione uma classe chamada `paragrafo-especial` ao segundo parágrafo. Usando a tag `<style>` no cabeçalho (apenas temporariamente para este exercício), mude a cor de todos os parágrafos para cinza, e a cor do `paragrafo-especial` para verde. Observe qual regra vence.

4. Exercício de fixação

Adicione um terceiro parágrafo ao seu HTML com o ID `aviso-importante` e também a classe `paragrafo-especial`. No CSS, crie uma regra para o ID definindo a cor do texto como laranja. Execute o código e reflita: por que o texto ficou laranja e não verde, se ambas as regras foram aplicadas?

2.2. Modelo de Caixa (Box Model)

1. Explicação teórica

Na web, absolutamente tudo é um retângulo. O **Box Model** (Modelo de Caixa) é o conceito fundamental de layout em CSS que define como os elementos são calculados em tamanho e espaçamento. Toda caixa CSS é composta por quatro áreas principais, de dentro para fora: o **Conteúdo** (Content), que é o texto ou imagem real; o **Preenchimento** (Padding), que é o espaço interno entre o conteúdo e a borda; a **Borda** (Border), que é a linha que delimita o elemento; e a **Margem** (Margin), que é o espaço invisível externo, separando este elemento dos demais ao seu redor. Compreender o Box Model é crucial para que os elementos não fiquem colados uns nos outros ou extrapolem o tamanho desejado na tela.

2. Demonstração prática

```
CSS
.cartao-produto {
  width: 300px;      /* Largura do conteúdo */
  padding: 20px;     /* Espaço interno (afasta o texto da borda) */
  border: 2px solid gray; /* Borda cinza de 2 pixels */
  margin: 15px;      /* Espaçamento externo (afasta dos outros cartões) */
}
```

Explicação linha por linha:

- Linha 2: Define que a largura da área de conteúdo do cartão será exatamente de 300 pixels.
- Linha 3: Adiciona 20 pixels de respiro dentro do cartão, em todos os quatro lados.
- Linha 4: Cria uma linha sólida, cinza, de 2 pixels de espessura ao redor do padding.
- Linha 5: Empurra qualquer outro elemento que esteja ao redor do cartão para 15 pixels de distância.

3. Exercício guiado

Crie uma `<div>` no seu HTML com a classe `caixa-teste`. Adicione nela a palavra "SENAI". No seu CSS, estilize essa classe aplicando um `background-color: lightblue`. Em seguida, aplique um `padding` de 30px, um `border` de 5px solid black e uma `margin` de 50px. Atualize a página e inspecione o elemento clicando com o botão direito para visualizar as caixas coloridas no navegador.

4. Exercício de fixação

Crie duas `<div>` idênticas usando a classe `caixa-teste` uma embaixo da outra. Ajuste a `margin-bottom` da primeira caixa para 20px e a `margin-top` da segunda caixa para 30px. Observe a distância final entre elas (fenômeno de *Margin Collapse*).

2.3. CSS Externo e Identidade Visual (Design System)

1. Explicação teórica

Embora seja possível escrever CSS dentro do próprio HTML, no mercado de trabalho e em projetos reais, nós utilizamos o **CSS Externo**. Isso significa criar um arquivo `.css` separado e "linká-lo" ao HTML. A principal vantagem é a reutilização: várias páginas HTML podem beber da mesma fonte de estilos. Além disso, as equipes de desenvolvimento definem um **Design System**, que é um conjunto padronizado de regras visuais do produto, garantindo consistência. Utilizamos variáveis CSS (declaradas no `:root`) para guardar a paleta de cores (3 a 4 cores principais), fontes e tamanhos. Dessa forma, se a empresa mudar a cor da marca no futuro, o desenvolvedor altera apenas uma variável, e todo o sistema é atualizado automaticamente.

2. Demonstração prática

No HTML (`index.html`):

HTML

```
<head>
  <link rel="stylesheet" href="style.css">
</head>
```

No CSS (`style.css`):

CSS

```
/* Definindo o Design System no escopo global */
```

```
:root {
  --cor-primaria: #0056b3;
  --cor-fundo: #f4f4f9;
  --fonte-padrao: 'Arial', sans-serif;
}
```

```
body {
  background-color: var(--cor-fundo);
  font-family: var(--fonte-padrao);
}
```

```
.botao-acao {
  background-color: var(--cor-primaria);
  color: white;
}
```

Explicação linha por linha:

- HTML Linha 3: A tag `<link>` conecta o arquivo `style.css` ao documento HTML atual.
- CSS Linha 2-6: A pseudo-classe `:root` guarda nossas variáveis globais (sempre começam com `--`).
- CSS Linha 9-10: O corpo da página utiliza as variáveis criadas para definir a cor de fundo e a fonte.
- CSS Linha 13-14: O botão aplica a cor primária dinâmica, mantendo a identidade visual do produto.

3. Exercício guiado

Crie um arquivo chamado `style.css` na mesma pasta do seu HTML. Transfira todo o código `<style>` que fizemos nos passos anteriores para este novo arquivo e delete a tag `<style>` do HTML. Em seguida, adicione a tag `<link rel="stylesheet" href="style.css">` no `<head>` do seu HTML e teste para ver se os estilos continuam funcionando.

4. Exercício de fixação

No topo do seu `style.css`, declare o `:root` e crie três variáveis de cores para a sua equipe: `--cor-primaria`, `--cor-secundaria` e `--cor-alerta`. Aplique essas variáveis na cor de fundo de diferentes elementos na sua página para testar a funcionalidade.

3. Demonstração Aplicada

Vamos visualizar a construção de um pequeno componente de interface, muito comum nas aplicações modernas: um cartão de perfil de usuário da aplicação.

HTML (`index.html`):

```
HTML
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <link rel="stylesheet" href="style.css">
  <title>Cartão de Usuário</title>
</head>
<body>
  <div class="card-usuario">
    <h2 class="titulo-usuario">João Silva</h2>
    <p class="funcao-usuario">Desenvolvedor Front-end</p>
    <button class="btn-contato">Entrar em contato</button>
  </div>
</body>
</html>
```

CSS (`style.css`):

```
CSS
:root {
  --brand-color: #2c3e50;
  --accent-color: #e74c3c;
  --bg-color: #ecf0f1;
  --font-base: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
}

body {
  background-color: var(--bg-color);
  font-family: var(--font-base);
  display: flex;
  justify-content: center;
  padding-top: 50px;
}

.card-usuario {
  background-color: white;
  width: 300px;
  padding: 30px;
  border-radius: 10px;
  box-shadow: 0 4px 8px rgba(0,0,0,0.1);
  text-align: center;
}
```

```
.titulo-usuario {
  color: var(--brand-color);
  margin-bottom: 5px;
}

.funcao-usuario {
  color: gray;
  margin-bottom: 20px;
}

.btn-contato {
  background-color: var(--accent-color);
  color: white;
  padding: 10px 20px;
  border: none;
  border-radius: 5px;
  cursor: pointer;
}

.btn-contato:hover {
  background-color: #c0392b;
}
```

Resultado esperado: Uma página centralizando um cartão com fundo branco, cantos arredondados, leve sombra, um título elegante, texto de função e um botão vermelho (Accent color) que escurece ao passar o mouse. Toda a manutenção visual fica no `style.css` e pode ser facilmente ajustada pelas variáveis no `:root`.

4. Casos Reais do Mercado

- **E-commerces (Mercado Livre, Amazon):** Imagine um site que possui milhares de botões de "Comprar". A equipe nunca estilizaria botão por botão. Eles criam um `style.css` rigoroso utilizando variáveis no `:root` e classes reaproveitáveis como `.btn-primario`. Se o marketing decidir mudar a cor da campanha na Black Friday, o programador altera apenas a variável `--cor-btn-primario` e o site inteiro é atualizado em segundos.
- **Aplicações Web e o Modo Escuro (Dark Mode):** Sistemas como o WhatsApp Web ou o YouTube permitem alternar entre Tema Claro e Tema Escuro. Isso é feito amplamente no mercado manipulando as variáveis CSS. O HTML permanece exatamente o mesmo, e o CSS simplesmente altera as cores de fundo e fonte guardadas nas variáveis dependendo da preferência do usuário.

5. FAQ – Dúvidas Reais de Alunos

1. Qual a diferença principal entre `margin` e `padding`?

O `padding` é o enchimento interno do elemento (aumenta o tamanho da caixa empurrando o conteúdo para o centro). A `margin` é o campo de força externo (empurra os outros elementos para longe sem alterar o tamanho da caixa atual).

2. Por que minhas regras de CSS não estão funcionando na página?

Verifique três coisas principais: se o arquivo `style.css` está na mesma pasta (ou se o caminho no

`<link>` está correto); se você não esqueceu de salvar o arquivo (Ctrl+S); e se não há um problema de *Especificidade* onde uma classe ou ID concorrente está vencendo sua regra atual.

3. Devo estilizar usando Tag, Classe ou ID?

A melhor prática no mercado é usar **Classes**. Estilizar direto na tag (`h1 { }`) pode quebrar o site todo se você usar o `h1` para finalidades diferentes. Usar ID (`#meu-botao`) gera muita rigidez e dificulta o reuso. Classes (`.btn-padrao`) garantem reusabilidade e a especificidade ideal.

4. Como escolho uma paleta de cores harmoniosa para o meu projeto?

Como programadores, frequentemente recebemos isso de um Designer (UI/UX). Porém, quando estamos criando do zero, recomendamos ferramentas gratuitas como o *Adobe Color* ou *Coolors.co* para extrair de 3 a 4 cores que farão parte do seu `:root`.

5. Posso colocar todo o CSS no arquivo HTML com a tag `<style>` e esquecer o arquivo externo?

Tecnicamente a página vai funcionar, mas não é recomendado profissionalmente. Misturar estruturação (HTML) com estilização (CSS) fere o princípio de separação de responsabilidades. Código espaguete fica difícil de manter, impossível de reaproveitar em outras páginas e afeta o cache de carregamento do navegador.

6. Desafio Prático

Contexto do problema:

O grupo de vocês está avançando no projeto semestral. A equipe precisa garantir que todos os formulários de login e os painéis de administração do sistema tenham exatamente a mesma cara (identidade visual).

Requisitos:

1. Reúnam a equipe e definam um **Design System** inicial: escolham uma paleta de 3 a 4 cores (primária, secundária, background e alerta), e escolham uma família de fontes (ex: Arial ou Roboto).
2. Criem o arquivo obrigatório `style.css` no repositório de vocês.
3. Declarem o `:root` com todas essas variáveis.
4. Construam uma página HTML simples de Login contendo: Título, campos de e-mail e senha, e um botão "Entrar". Linkem o arquivo CSS a este HTML.
5. Utilizem Seletores de Classe e o Modelo de Caixa para aplicar margens, espaçamentos internos e as cores do design system na estrutura.

Resultado esperado:

Um repositório contendo um `index.html` e o `style.css` vinculado de forma limpa. A página de login não deve encostar nas bordas do navegador, o formulário deve parecer um cartão centralizado, e o botão deve utilizar a variável de cor primária da equipe.

Sugestões de solução:

Utilizem as propriedades `padding` dentro do formulário para dar respiro aos campos de texto, e apliquem `margin-bottom` aos inputs para que não fiquem colados uns aos outros. Garantam a leitura importando bem o arquivo: `<link rel="stylesheet" href="style.css">`.

7. Conexão com a Situação de Aprendizagem

Este conhecimento é fundamental para o projeto de "Desenvolvimento e Implantação de Soluções Digitais Inovadoras" que conduzimos neste semestre. A competência técnica (CT01) exige "reconhecer requisitos de qualidade, integridade, usabilidade...". O arquivo `style.css` externo que vocês acabaram de criar será o pilar da **usabilidade e qualidade** da aplicação de vocês. As variáveis globais e o padrão de classes estabelecido agora serão reutilizados quando implementarem as views geradas em PHP mais para frente. Ter um Front-end padronizado e limpo garante que o sistema atenda aos padrões exigidos de um software profissional, cumprindo diretamente com a entrega de um sistema com boa "Responsividade" e "Frontend".

8. Preparação para a Próxima Aula

Excelente trabalho hoje organizando a casa e dando cor ao projeto! Agora vocês sabem como separar a responsabilidade visual do HTML e como moldar o tamanho das caixas na tela. Na nossa próxima aula, vamos explorar **Layouts Flexíveis com Flexbox**. Vocês aprenderão como organizar esses cartões e formulários em colunas e linhas perfeitas, permitindo construir navegações dinâmicas e garantindo que o site funcione perfeitamente tanto no monitor do computador quanto na tela de um celular. Tragam seu `style.css` afiado!